

stack · v1.1.0

Full Technical Documentation

One skill — and now one standalone installer — to install them all.

stack installs [Longhand](#), [Context-Mode](#), and [Hardgate](#) as a coordinated Claude Code stack — with timestamped config backup, per-tool idempotency guards, and a hardened post-install verification checklist.

Table of Contents

1. What This Project Does
 2. Architecture Overview
 3. The Three Tools
 4. Installation — Option A: Standalone Installer
 5. Installation — Option B: /stack Claude Code Skill
 6. What Happens During Install
 7. Post-Install Verification
 8. Design Decisions
 9. File Reference
 10. Tested Versions
 11. Known Limitations
-

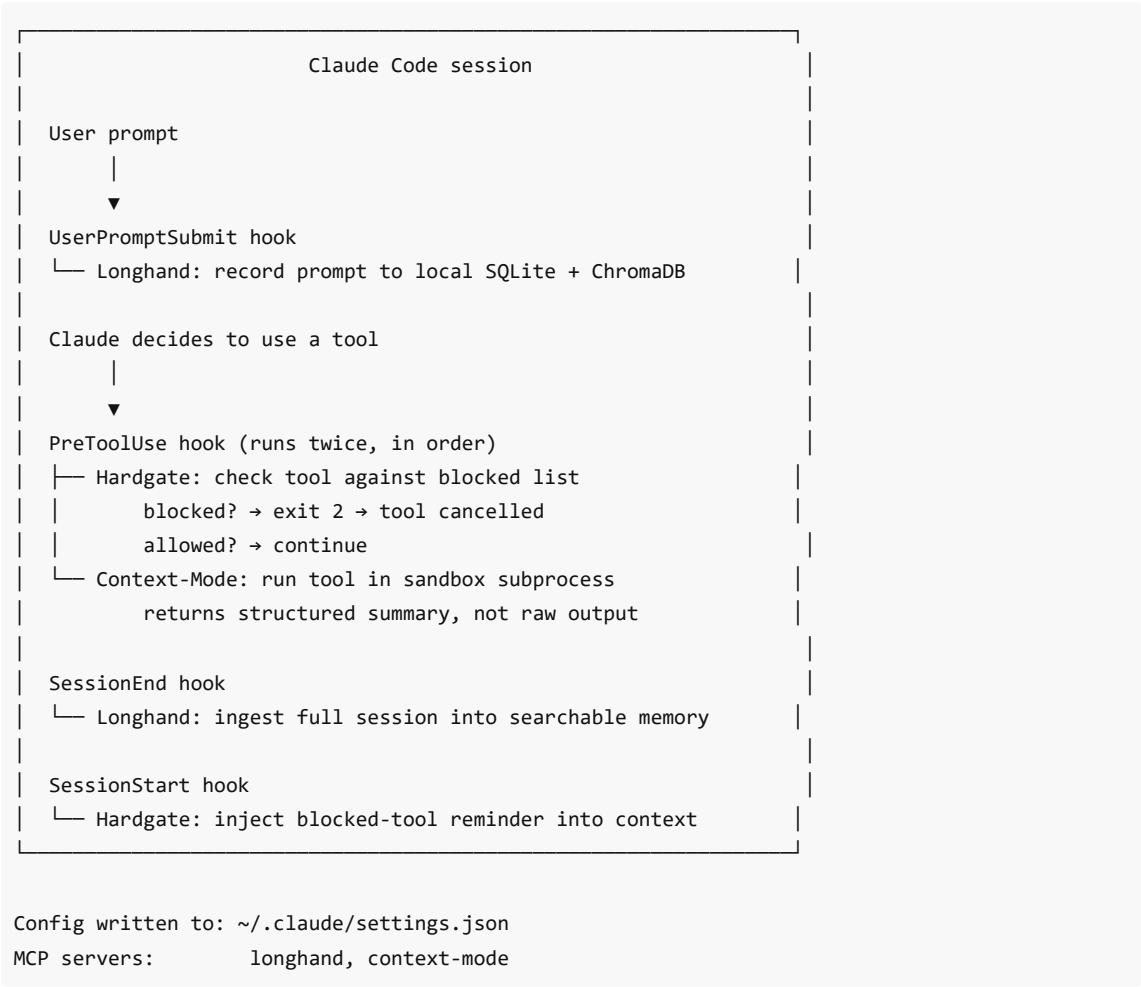
1. What This Project Does

Claude Code is a powerful coding assistant, but out of the box it has three significant gaps:

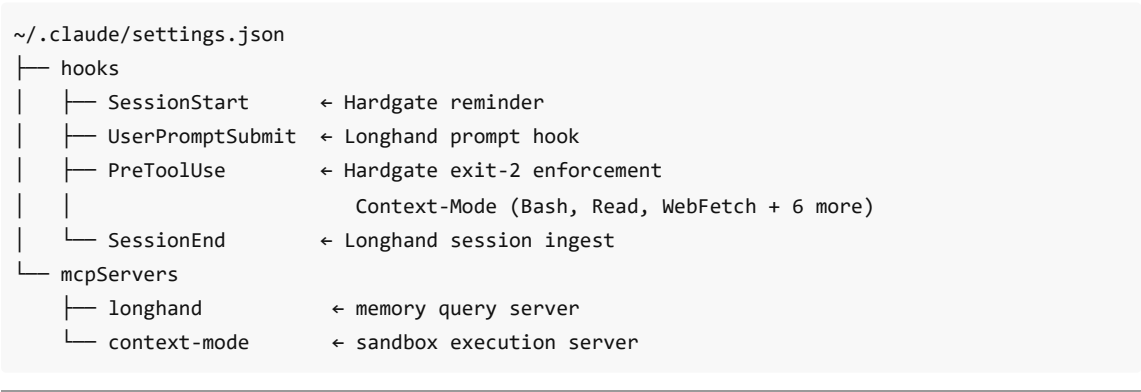
- **No memory** — every session starts from zero. Claude doesn't know what you worked on yesterday.
- **Context flooding** — raw tool output (file reads, bash output, web fetches) pours into the conversation and fills up Claude's context window, causing it to forget earlier parts of the session.
- **No enforcement layer** — Claude can call any tool it decides to use. There's no mechanism to require human approval before risky operations.

`stack` installs three community-built tools that fill exactly these gaps — in the right order, with the right wiring, verified after install.

2. Architecture Overview



Config File Layout



3. The Three Tools

Longhand

Author: [Wynelson94 · github.com/Wynelson94/longhand](https://github.com/Wynelson94/longhand)

Stores every Claude Code session verbatim in SQLite + ChromaDB. Enables semantic recall in ~126ms with zero API cost. Hooks into UserPromptSubmit (to capture prompts as they arrive) and SessionEnd (to ingest the full session after close).

Install command: `pip install longhand`

Artifacts installed:

- SessionEnd hook in `~/.claude/settings.json`
- UserPromptSubmit hook in `~/.claude/settings.json`
- MCP server entry for `longhand mcp-server`

Context-Mode

Author: [scottconverse · github.com/scottconverse/context-mode](https://github.com/scottconverse/context-mode)

Sandboxes tool output. Intercepts PreToolUse events for Bash, Read, WebFetch and six additional tool types, runs them in a subprocess sandbox, and returns a structured summary instead of raw output. Keeps raw command results off the context window.

Install command: `git clone + node install.js`

Artifacts installed:

- PreToolUse hooks (one per tool type) in `~/.claude/settings.json`
- MCP server entry for context-mode sandbox

Required matchers: Bash, Read, WebFetch (minimum — install adds more)

Hardgate

Author: [scottconverse · github.com/scottconverse/hardgate](https://github.com/scottconverse/hardgate)

Blocks forbidden tool calls via `exit 2` hooks. Requires one interactive configuration step (running `/hard-gate` inside a Claude Code session to choose which tools to enforce). After configuration, fires on every PreToolUse event.

Install method: Interactive — requires `/hard-gate` inside an active Claude Code session.

Artifacts installed:

- PreToolUse enforcement hook script
 - SessionStart reminder hook in `~/.claude/settings.json`
-

4. Installation — Option A: Standalone Installer (v1.1.0+)

The standalone installer runs without an open Claude Code session. It is the recommended install path.

Requirements

- Python 3.10+
- Node.js 18+
- Claude Code CLI (`claude` on PATH)
- Full `stack` repo cloned locally

Steps

macOS / Linux:

```
git clone https://github.com/scottconverse/stack
cd stack
bash install.sh
```

Windows (any terminal — no Git Bash required):

```
git clone https://github.com/scottconverse/stack
cd stack
python install.py
```

Or double-click `install.bat` .

What install.py Does

1. Locates `scripts/verify.py` relative to itself
2. Runs pre-flight checks (Python version, Node version, tool availability)
3. Takes timestamped snapshots of `~/.claude/settings.json` and `~/.claude.json`
4. Installs Longhand via `pip install longhand` + runs `longhand setup` + registers MCP server
5. Discovers Context-Mode via Python-native directory walk (no `find` dependency — works on Windows)
6. Installs Context-Mode via `git clone` + `node install.js`
7. Pauses for interactive Hardgate configuration — displays instructions, waits for user to run `/hard-gate` in a Claude Code session, then presses Enter
8. Imports `scripts/verify.py` via `importlib.util` and runs the full post-install check suite
9. Exits with the verifier's exit code (0 = all pass, 1 = check failures, 2 = config corruption + auto-restore)

Verify-Only Mode

```
python install.py --verify
```

Skips installation; runs the verifier only. Useful for checking state after manual changes.

5. Installation — Option B: /stack Claude Code Skill

Install from inside an active Claude Code session using the `/stack` skill command.

Requirements

- Same as Option A, plus:
- An active Claude Code session
- bash shell (Git Bash or WSL on Windows)
- `skills/stack.md` added to your Claude Code skill path

Steps

```
git clone https://github.com/scottconverse/stack
# Add skills/stack.md to Claude Code skill path
claude
/stack
```

The skill file instructs Claude Code to perform the same install sequence as `install.py`. Both paths share `scripts/verify.py` for identical post-install verification.

6. What Happens During Install

Phase 0 — Verify.py Discovery

Three-stage discovery: env var (`STACK_DIR`) → plugin cache → bounded home directory walk. Both `install.py` and the `/stack` skill use the same discovery logic.

Pre-flight

Checks Python 3.10+, Node 18+, and tool locations. Takes timestamped snapshots:

```
~/ .claude/settings.json.stack-backup-YYYYMMDD-HHMMSS
~/ .claude.json.stack-backup-YYYYMMDD-HHMMSS
```

Multiple runs produce independent backups — reruns do not overwrite prior snapshots.

Idempotency

Before installing each tool, the installer checks ALL required artifacts for that tool. Only skips if every artifact is present and correct. Partial installs are always re-run from scratch.

Install Sequence

1. **Longhand** — `pip install longhand` → `longhand setup` → MCP registration
2. **Context-Mode** — `clone` → `node install.js` (registers hooks and MCP server)
3. **Hardgate** — interactive pause → `/hard-gate` configuration in Claude Code session

Session State (skill path only)

The `/stack` skill persists `VERIFY_PY`, `STACK_TS`, and `CTX_DIR` to `~/ .claude/stack-session.env` before the Hardgate interactive pause so variables survive the interaction boundary.

7. Post-Install Verification

`scripts/verify.py` runs automatically after install. Run standalone at any time:

```
python scripts/verify.py
# or
python install.py --verify
```

Check	Level	Retry command
Longhand SessionEnd hook	Required	longhand setup
Longhand UserPromptSubmit hook	Required	longhand prompt-hook install
Context-Mode PreToolUse hooks	Required	node install.js (from context-mode dir)
Longhand MCP server	Required	claude mcp add longhand -s user -- longhand mcp-server
Context-Mode MCP server	Required	node install.js (from context-mode dir)
Hardgate enforcement	Warning only	/hard-gate inside Claude Code

Exit Codes

Code	Meaning	Action
0	All required checks pass	Restart Claude Code
1	One or more checks failed; config intact	Follow retry commands
2	Config file malformed	Both files auto-restored from backup

The exit-2 restore uses symmetric logic: both `settings.json` and `.claude.json` are restored if either is malformed. The restore hint shows the most recent backup path via `ls -t ... | head -1`.

8. Design Decisions

Why a standalone installer (`install.py`) alongside the skill? The `/stack` skill requires an open Claude Code session — which means the user already has Claude Code configured and working. For new setups or machine migrations where Claude Code isn't running yet, a standalone installer removes that dependency entirely.

Why Python stdlib only in `install.py` ? No additional packages to install before you can run the installer. Reduces friction to zero on a fresh machine.

Why Python-native context-mode discovery (no `find`)? `find` is a Unix command that doesn't exist on Windows without Git Bash or WSL. A Python-native `os.walk` with a depth limit works identically on all platforms.

Why share `verify.py` between both install paths? Single source of truth for what "correctly installed" means. Both paths import the same module via `importlib.util.spec_from_file_location` , so a fix to the verifier benefits both paths immediately.

Why `exit 2` for config corruption (not `exit 1`)? Exit 1 signals "checks failed but config is intact — retry." Exit 2 signals "config is damaged — auto-restore has run." The distinction lets callers handle the two cases differently.

Why timestamped backups (not a single `.bak` file)? Reruns must not silently overwrite a prior backup. If an install corrupts config and the user re-runs before noticing, the original pre-install state must still be recoverable.

9. File Reference

File	Purpose
<code>install.py</code>	Standalone installer — Python stdlib only
<code>install.bat</code>	Windows double-click launcher for <code>install.py</code>
<code>install.sh</code>	macOS/Linux launcher for <code>install.py</code>
<code>skills/stack.md</code>	Claude Code skill file for <code>/stack</code> command
<code>scripts/verify.py</code>	Post-install state verifier; shared by both install paths
<code>tests/test_verify.py</code>	29 unit tests covering all verifier logic and exit codes
<code>docs/index.html</code>	GitHub Pages landing page
<code>docs/USER-MANUAL.html</code>	Plain-language user manual
<code>docs/README-FULL.pdf</code>	This document

10. Tested Versions

Tool	Tested version
Longhand	0.5.5
Context-Mode	1.6.0
Python	3.14.3
Node.js	18+

Pin Longhand if you need a reproducible install:

```
pip install longhand==0.5.5
```

Context-Mode is installed from a local clone — pin by checking out the validated commit hash before running `node install.js` .

11. Known Limitations

Config presence $\neq$ MCP liveness. The verifier checks that config entries are present and correctly structured. It does not verify that MCP servers are actually running and responding. After install, always confirm runtime health:

```
longhand doctor
claude mcp list
```

Hardgate requires a live Claude Code session. The `/hard-gate` configuration step cannot be automated. Both install paths pause and wait for the user to complete it interactively.

Context-Mode discovery depth. The Python-native walk in `install.py` is bounded to 5 directory levels from the home directory. If you've cloned Context-Mode unusually deep in your directory tree, set `CONTEXT_MODE_DIR` to its absolute path before running the installer.